

TaskFlow Local Manual Test Guide

Step-by-step Ubuntu commands to run the TaskFlow setup locally without creating Docker containers, Docker images, Kubernetes manifests, or DevOps assets.

Assumptions: you are on Ubuntu 22.04 or 24.04, the project lives at ~/taskflow or another local path, and required packages are not installed yet. Replace paths and passwords only where noted.

Component	Local port	Purpose
PostgreSQL	5432	Persistent storage for task-service and notification-service
Redis	6379	Task read cache for task-service
Kafka	9092	Task event broker
task-service	8080	Spring Boot REST API
notification-service	8081	Kafka consumer and notification recorder
frontend	5173	React UI

1. Install Required Packages

Install Java 21, Maven, Node.js, npm, PostgreSQL, Redis, curl, jq, and helper tools.

```
sudo apt update
sudo apt install -y openjdk-21-jdk maven postgresql postgresql-contrib redis-server \
  curl jq wget tar ca-certificates gnupg lsb-release

java -version
mvn -version
node -v || true
npm -v || true
psql --version
redis-server --version
```

If Node.js is missing or too old, install the current LTS from NodeSource. This project was verified with modern Node 20+ behavior.

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
node -v
npm -v
```

2. Install Kafka Locally

Install Kafka into /opt/kafka and create a convenience symlink. This runs directly on your host, not in Docker.

```
cd /tmp
wget https://archive.apache.org/dist/kafka/3.9.0/kafka_2.13-3.9.0.tgz
sudo tar -xzf kafka_2.13-3.9.0.tgz -C /opt
sudo ln -sfn /opt/kafka_2.13-3.9.0 /opt/kafka
sudo chown -R "$USER":"$USER" /opt/kafka_2.13-3.9.0
/opt/kafka/bin/kafka-storage.sh --help >/dev/null
```

3. Prepare The Project Directory

Move into your TaskFlow project root. Adjust this path if you cloned or copied it elsewhere.

```
cd ~/taskflow

# Quick structure check
ls
ls frontend task-service notification-service database devops docs

# devops should exist but remain empty for now
find devops -mindepth 1 -maxdepth 1 -print
```

4. Start PostgreSQL And Create Databases

Start PostgreSQL, create an application user, and create separate databases for the two services.

```
sudo systemctl enable --now postgresql
sudo systemctl status postgresql --no-pager

sudo -u postgres psql <<'SQL'
DO $$
BEGIN
    IF NOT EXISTS (SELECT FROM pg_roles WHERE rolname = 'taskflow') THEN
        CREATE ROLE taskflow LOGIN PASSWORD 'taskflow';
    END IF;
END
$$;
SQL

sudo -u postgres createdb -O taskflow taskflow || true
sudo -u postgres createdb -O taskflow taskflow_notifications || true

PGPASSWORD=taskflow psql -h localhost -U taskflow -d taskflow -c "select current_database();"
PGPASSWORD=taskflow psql -h localhost -U taskflow -d taskflow_notifications -c "select current_database();"
```

5. Start Redis

Run Redis as a normal Ubuntu service and verify that it responds.

```
sudo systemctl enable --now redis-server
sudo systemctl status redis-server --no-pager
redis-cli ping
```

6. Start Kafka In KRaft Mode

Open a dedicated terminal for Kafka. The commands below format a local Kafka data directory once, then start the broker.

```
mkdir -p ~/taskflow-runtime/kafka-logs

KAFKA_CLUSTER_ID="$(/opt/kafka/bin/kafka-storage.sh random-uuid)"
/opt/kafka/bin/kafka-storage.sh format \
  -t "$KAFKA_CLUSTER_ID" \
  -c /opt/kafka/config/kraft/server.properties \
  --ignore-formatted

/opt/kafka/bin/kafka-server-start.sh /opt/kafka/config/kraft/server.properties
```

In another terminal, create and verify the task event topic.

```
/opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 \
  --create --if-not-exists --topic task-events --partitions 1 --replication-factor 1

/opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --list
```

7. Run task-service

Open a new terminal from the project root and start the REST API. Flyway will create the tasks table.

```
cd ~/taskflow/task-service

export SERVER_PORT=8080
export SPRING_DATASOURCE_URL=jdbc:postgresql://localhost:5432/taskflow
export SPRING_DATASOURCE_USERNAME=taskflow
export SPRING_DATASOURCE_PASSWORD=taskflow
export SPRING_DATA_REDIS_HOST=localhost
export SPRING_DATA_REDIS_PORT=6379
export SPRING_KAFKA_BOOTSTRAP_SERVERS=localhost:9092
export TASKFLOW_KAFKA_TASK_EVENTS_TOPIC=task-events

mvn spring-boot:run
```

Health check from a separate terminal:

```
curl -s http://localhost:8080/actuator/health | jq
curl -s http://localhost:8080/actuator/prometheus | head
```

8. Run notification-service

Open another terminal from the project root and start the Kafka consumer. Flyway will create the notifications table.

```
cd ~/taskflow/notification-service

export SERVER_PORT=8081
export SPRING_DATASOURCE_URL=jdbc:postgresql://localhost:5432/taskflow_notifications
export SPRING_DATASOURCE_USERNAME=taskflow
export SPRING_DATASOURCE_PASSWORD=taskflow
export SPRING_KAFKA_BOOTSTRAP_SERVERS=localhost:9092
export TASKFLOW_KAFKA_TASK_EVENTS_TOPIC=task-events

mvn spring-boot:run
```

Health check from a separate terminal:

```
curl -s http://localhost:8081/actuator/health | jq
curl -s http://localhost:8081/actuator/prometheus | head
```

9. Run The Frontend

Open another terminal and start the React app.

```
cd ~/taskflow/frontend

export VITE_TASK_API_BASE_URL=http://localhost:8080
npm install
npm run dev
```

Open the UI at <http://localhost:5173>.

10. End-To-End API Smoke Test

Create a task, list tasks, complete the task, then confirm that notification-service recorded Kafka events.

```
TASK_ID=$(curl -s -X POST http://localhost:8080/api/tasks \
-H 'Content-Type: application/json' \
-d '{"title":"Manual local E2E","description":"PostgreSQL Redis Kafka Spring React"}' \
| jq -r '.id')

echo "$TASK_ID"

curl -s http://localhost:8080/api/tasks | jq

curl -s -X PATCH "http://localhost:8080/api/tasks/${TASK_ID}/complete" | jq

sleep 3

PGPASSWORD=taskflow psql -h localhost -U taskflow -d taskflow \
-c "select id, title, status, created_at, completed_at from tasks order by created_at desc limit 5;"

PGPASSWORD=taskflow psql -h localhost -U taskflow -d taskflow_notifications \
-c "select task_id, event_type, status, received_at from notifications order by received_at desc limit 10;"
```

11. Run Automated Tests

Run the project tests locally. Backend tests require Maven; frontend tests require npm dependencies.

```
cd ~/taskflow/task-service
mvn test

cd ~/taskflow/notification-service
mvn test

cd ~/taskflow/frontend
npm test
npm run build
```

12. Optional Manual Kafka Inspection

Watch task events directly from Kafka while you create or complete tasks.

```
/opt/kafka/bin/kafka-console-consumer.sh \
--bootstrap-server localhost:9092 \
--topic task-events \
--from-beginning
```

13. Clean Shutdown

Stop app terminals with Ctrl+C. Then stop host services if you no longer need them.

```
sudo systemctl stop redis-server
sudo systemctl stop postgresql

# Stop Kafka with Ctrl+C in the Kafka terminal.
```

14. Troubleshooting Checklist

```
# Port checks
ss -ltnp | grep -E '':5432|:6379|:9092|:8080|:8081|:5173'

# PostgreSQL connection
PGPASSWORD=taskflow psql -h localhost -U taskflow -d taskflow -c '\dt'

# Redis connection
redis-cli ping

# Kafka topic state
/opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --describe --topic task-events

# Spring health endpoints
curl -s http://localhost:8080/actuator/health | jq
curl -s http://localhost:8081/actuator/health | jq

# If Kafka data is corrupted during practice, stop Kafka and reset local logs
rm -rf /tmp/kraft-combined-logs ~/taskflow-runtime/kafka-logs
```